TECHNICAL ELECTIVE 4

WEB-BASED SCHOOL VISITOR LOG MANAGEMENT SYSTEM

Whole System Concept and Proposed Design

Prepared by:

Submitted to:

2026

1. System Overview

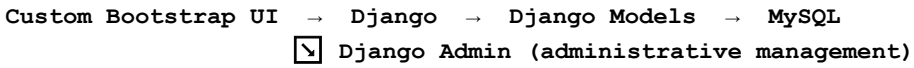
The Web-Based School Visitor Log Management System is a Django-based application designed to replace or support the traditional paper visitor logbook used at school entrances. It focuses on non-student visitors such as parents or guardians, alumni, delivery personnel, contractors, government personnel, invited guests, and other individuals who enter the school for an official or personal purpose.

Authorized school personnel will record a visitor's information, purpose, destination, time-in, and time-out. The system will also show who is currently inside the school, maintain searchable visitor history, and provide basic visitor statistics. The goal is to make visitor records easier to organize, retrieve, and monitor while keeping the project realistic for the course timeline.

2. Proposed Technology Stack

Component	Technology	Purpose
Development Environment	Visual Studio Code	Main code editor/development environment.
Programming Language	Python	Server-side programming language used by Django.
Web Framework	Django	Handles URLs, views, models, forms, templates, authentication, and application logic.
Frontend	HTML5 + CSS3	Provides page structure and custom styling.
UI Framework	Bootstrap 5	Creates responsive forms, cards, navigation, tables, and dashboard layouts.
Database	MySQL	Stores users, visitors, offices, and visit records.
Administrative Interface	Django Admin	Built-in interface for authorized administrative data management; it is not the database itself.
Version Control	Git	Tracks source-code changes.
Repository	GitHub	Stores the project repository and commit history.

Important distinction: MySQL is the actual database. Django models define the application data structure, Django migrations create or update database tables, and Django Admin provides a convenient interface for managing selected records.



3. Target Users and Roles

User	Main Responsibilities
Administrator	Manages authorized user accounts and school offices/destinations; can view visitor records, active visits, history, dashboard information, and reports.
Security Personnel / Authorized Staff	Registers visitors, records visit information and time-in, records time-out, monitors active visitors, and searches visitor history.

User	Main Responsibilities
Visitor	Provides the required visitor and visit information. The visitor does not need a system account.

4. Visitor Information to be Recorded

- First name and last name
- Contact number
- Address
- Visitor type (for example: Parent/Guardian, Alumni, Delivery Personnel, Contractor, Government Personnel, Guest, or Other)
- Purpose of visit
- School office or destination
- Person to visit, when applicable
- Date and time-in
- Time-out
- Visit status: Inside or Completed
- Authorized staff member who recorded the visit

5. Core System Features

Feature	Description
User Authentication	Authorized administrators and staff can securely log in and log out.
Visitor Registration	Staff can create or locate a visitor record and enter the information needed for the visit.
Visit and Time-In Recording	The system records the visitor's purpose, destination, person to visit, and arrival time.
Visitor Time-Out	Staff can record departure time and change the visit status from Inside to Completed.
Active Visitor Monitoring	A page shows visitors who are currently inside the school premises.
Visitor History	Authorized users can review previous visit records.
Search and Filtering	Records can be searched or filtered by visitor name, date, visitor type, destination, or status.
Dashboard and Basic Reports	The dashboard displays useful counts and summaries such as Visitors Today, Currently Inside, Completed Today, visitor trends/types, recent visits, and top school destinations.

6. Proposed Dashboard and UI Direction

The interface will use the dashboard reference selected by the group as visual inspiration only. The final interface should not copy the reference exactly. It will use a similar information hierarchy—sidebar navigation, summary cards, charts, and a recent-visits table—but will be redesigned for the school visitor workflow.

Dashboard Area	Proposed School Version

Dashboard Area	Proposed School Version
Summary Cards	Visitors Today, Currently Inside, Completed Today, and This Month or Peak Day.
Visitor Trend	Daily, weekly, or monthly visitor counts.
Visitor Type Chart	Parent/Guardian, Alumni, Delivery, Contractor, Government Personnel, Guest, and Other.
Recent Visits	Visitor, destination, time-in, time-out/status, and related information.
Top Destinations	School offices receiving the highest number of visitors, such as Registrar, Dean's Office, Cashier, or Guidance Office.
Sidebar	Dashboard, Register Visitor, Active Visitors, Visitor History, Reports; administrator-only options for Users and School Offices; Logout.

7. Proposed Main Screens

1. Login Page – username and password fields for authorized users.

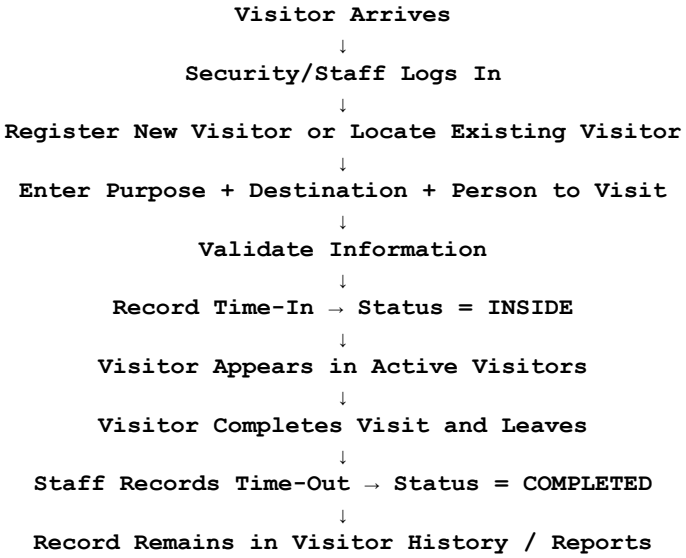
2. Dashboard – visitor statistics, charts, recent visits, top destinations, and quick access to visitor registration.

3. Register Visitor – visitor details, visitor type, purpose, destination, person to visit, and Time In/Register action.

4. Active Visitors – list of visitors currently inside the school with search and Time Out action.

5. Visitor History / Reports – searchable and filterable previous visit records with basic reporting information.

8. Basic Visitor Workflow

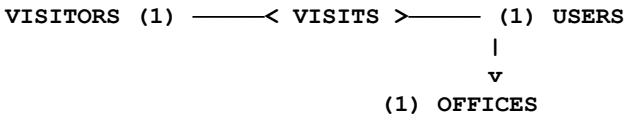


9. Proposed Database Concept

The initial database can remain small and understandable. Four principal entities are enough for the first version: Users, Visitors, Offices, and Visits.

Entity	Important Fields	Relationship / Purpose

Entity	Important Fields	Relationship / Purpose
Users	id, username, password, first_name, last_name, role	Authorized administrators/staff. One user can record many visits.
Visitors	visitor_id, first_name, last_name, contact_number, address, visitor_type	Stores visitor identity/contact information. One visitor can have many visits over time.
Offices	office_id, office_name, location	Stores selectable school destinations. One office can receive many visits.
Visits	visit_id, visitor_id, office_id, recorded_by, purpose, person_to_visit, time_in, time_out, status	Connects the visitor, destination, and staff member for each individual visit.



10. Scope

The system will focus on digitally recording and monitoring non-student visitors entering and leaving the school premises. It will include authentication for authorized users, visitor registration, visit and time-in/time-out recording, active visitor monitoring, visitor history, search and filtering, basic dashboard statistics and reports, and management of authorized users and school offices/destinations. The application will be web-based and designed to remain usable on common desktop and mobile screen sizes.

11. Limitations

The system will not replace student attendance, employee attendance, enrollment, grading, or physical access-control systems. Visitors will not be required to create personal accounts. The initial version will not include biometric identification, facial recognition, RFID/NFC gates, automated government-ID verification, or automated background checking. The accuracy of visitor records will depend on the information provided by visitors and entered by authorized personnel.

12. Example Scenario

A parent named Maria Santos arrives at the school to request records from the Registrar's Office. The security personnel logs in, searches whether Maria already has a visitor profile, and creates one if necessary. The staff selects Parent/Guardian as the visitor type, records "Request school records" as the purpose, selects Registrar's Office as the destination, and records the time-in. Maria then appears on the Active Visitors page with the status Inside. When she leaves, the staff clicks the Time Out action. The system records her departure time, changes the visit status to Completed, and keeps the record in Visitor History for later searching and reporting.

13. Development Approach

1. Set up Python, Django, MySQL, Git, and the Django project in Visual Studio Code.
2. Configure Django to use the local MySQL database instead of the default SQLite database.
3. Create Django models for visitors, offices, and visits; use Django authentication for authorized users.
4. Register appropriate models in Django Admin for administrative management.

5. Build the custom Bootstrap interface beginning with Login, Dashboard, Register Visitor, Active Visitors, and Visitor History.

6. Implement validation, search/filtering, time-in/time-out, and role restrictions.

7. Test CRUD operations and database persistence, then push the project to GitHub.

8. Prepare the application for the deployment requirements later in the course.

14. Deliverable 1 Design Outputs

For Deliverable 1, the concept should be represented through a Use Case Diagram, ERD, database table design, at least four UI wireframes, and a system workflow. The five proposed wireframes are Login, Dashboard, Register Visitor, Active Visitors, and Visitor History/Reports.

15. Final Concept Summary

In summary, the project is a manageable Django + MySQL school visitor-management application centered on a simple real-world workflow: register the visitor, record the reason and destination, time the visitor in, monitor who is currently inside, time the visitor out, and preserve the visit in searchable history. Django Admin will support administrative data management, while the actual day-to-day visitor workflow will use a custom responsive Bootstrap interface. The first version deliberately avoids advanced hardware and identity-verification features so the group can focus on a complete, working, and well-designed core system.